Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Regional Transportation District

RTD Denver EMU

Project # 7230

Door Control Unit Application

Master Software Design Description (SDD)

Contract Number	-	CDRL Number	-
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim Software Designer	James Kim 2011-10-05
Verified :	Martin Ouellet, Eng. Software Department Supervisor	2011-10-06
Approved :	Aldo Di Rocco Project Manager	for Aldo Di Rocco 2011-10-07

Document No.	**201111450**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-05**

Revision List

Rev.	Date	By	ECD	Description of changes
1	2011-10-05	James Kim	-	Official release.
0.1	2011-09-27	James Kim	-	Implementing comments from internal inspection I-1851
0	2011-09-27	James Kim	-	Original issue.

Preface

This Software Design Description (SDD) complies with the requirements of IEEE Std. 1016-1998 and IEEE Std. 1558-2004.

Table of Contents

Revision List 2
Preface 2
Table of Contents 3
List of Figures 4
List of Tables 4

1 INTRODUCTION 5

1.1 Purpose 5
1.2 Scope 5
1.3 Definitions, Acronyms and Abbreviations 6
1.3.1 Definitions 6
1.3.2 Acronyms and Abbreviations 6
1.4 Document Organization 7

2 REFERENCES 8

3 SOFTWARE DESIGN 9

3.1 Context Overview 9
3.2 Decomposition Description 10
3.2.1 Door Control Unit Application Main 10
3.2.2 Self-Test Module 10
3.2.3 Monitoring Module 10
3.2.4 Watchdog Module 10
3.2.5 Door Operator Module 10
3.2.6 Portable Test Application Protocol Module 11
3.2.7 Real-Time Data Monitoring (RTDM) Module 11
3.2.8 Fault Logger Module 11
3.2.9 IO Control Module 11
3.2.10 PID Module 11
3.2.11 Serial Port Driver 11
3.2.12 Real-Time Clock 11
3.2.13 SPI Module 11
3.3 Data Decomposition 12
3.4 Dependency Description 12
3.4.1 Inter-Entity Dependencies 12
3.4.2 Inter-Process Dependencies 12
3.5 Data Dependencies 13
3.6 Interface Description 13
3.6.1 Internal Interfaces 13
3.6.2 External Interfaces 13
3.6.3 Data Dictionary 13
3.7 Detailed Design 13
3.7.1 BootTask() Function 14
3.7.2 main() Function 15
3.8 Operating System Services 16
3.9 Error / Warning Handling 16

List of Figures

Figure 3-1: Hardware Block Diagram 9
Figure 3-2: Hierarchical Diagram 10
Figure 3-3: Inter-Process Dependencies Diagram 12
Figure 3-4 -- BootTask() Function Algorithm 14
Figure 3-5 -- main() Function Algorithm 15

List of Tables

Table 1-1: Software Modules 5

1 INTRODUCTION

1.1 Purpose

The purpose of this document is to describe the architecture of the Door Control Unit (DCU) Application developed for Regional Transportation District.

The DCU Application is designed to control doors and communicate with Neuron Gateway Application (NGA) to report its status and health to the Car Network. The communication link between the DCU Application and the NGA is the Serial Peripheral Interface (SPI).

The intended audience of this document is the Vapor Stone Rail Systems (VSRS) software engineering group, the car builder software engineering group and ultimately, the transit authority software engineering group.

1.2 Scope

The scope of this document covers the architecture design of the Door Control Unit Application. This design includes:

- Interface to the door control train/car lines
- Control of the door operator and motor
- Monitoring of DCU signals
- Interface to local crew switches
- Interface to cut-out device
- Control of the indicator light and alarm sound devices
- Interface to the Neuron Gateway Application

Communication with the Car Network and interface with the Portable Test Unit (PTU) Application are outside the scope of this document. Please refer to the Neuron Gateway Application SDD and the PTU Application SDD for their design descriptions.

This SDD documents only the architecture portion of the design and the detailed design of the main functions. Detailed design of other modules is included in separate documents as indicated in the Table 1-1 below.

Table 1-1: Software Modules

Module	Type	Vapor Document
Main	Specific	This document
Door Operator	Generic	201111568
Fault Logger	Generic	201111583
IO Control	Specific	201111628
Monitoring	Specific	201111630
PID	Generic	201111554
Portable Test Application Protocol	Generic	201111586
Real-Time Clock (RTC)	Generic	201111581
Real-Time Data Monitoring (RTDM)	Generic	201111585
Self-Test	Generic	201111584
Serial Driver	Generic	201111582
Watchdog	Generic	201111544

1.3 Definitions, Acronyms and Abbreviations

1.3.1 Definitions

Door Control Unit: A processor based electronic unit running software that controls the door, provides monitoring functions, and interacts with the user via a PTU Application and a network.

Event: Term use to globally identify an Info, an Exception or a Failure (see Section 1.3.1.1 below).

1.3.1.1 Event Level Definitions by Increasing Importance

Info: An Info is an occurrence of an informative condition monitored by the system, which is neither an exception nor a failure.

Exception: An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).

Failure: A failure is an occurrence of a condition that is outside of the specified parameters and affecting this LRU. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.3.2 Acronyms and Abbreviations

Acronym	Meaning
ARTX166	(Keil) Advanced Real-Time Operating System for C166
BL	Boot Loader
BLA	Boot Loader Application
CPU	Central Processing Unit
DCU	Door Control Unit
DOP	Door Operator
FLOG	Fault Logger
ICD	Interface Control Document
ID	Identification
IEEE	Institute of Electrical and Electronics Engineers, Inc.
IO	Input/Output
LDTS	Local Diagnostic and Test System
LRU	Line Replaceable Unit
NGA	Neuron Gateway Application
NVRAM	Non-Volatile Random Access Memory
OS	Operation System
PID	Proportional Integral Derivative
PROM	Programmable Read-Only Memory
PTE	Portable Test Equipment
PTU	Portable Test Unit
PWM	Pulse Width Modulation
RAM	Random Access Memory
ROM	Read-Only Memory
RTC	Real-Time Clock
RTD	Regional Transportation District
RTDM	Real-Time Data Monitoring
SACS	Software Applications Configuration Specifications
SCI	Software Configuration Item
SDD	Software Design Description
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
SRS	System Requirements Specification
VSRS	Vapor Stone Rail Systems Inc.

1.4 Document Organization

- Section 1, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, lists the factors that affect the product and its requirements.
- Section 3, summarizes the followings:
    - Section 3.1, Context Overview: provides the system's context where each software application is running,
    - Section 3.2, Decomposition Description: describes software entities,
    - Section 3.3, Data Decomposition: gives a description of the data entities used ,
    - Section 3.4, Dependency Description: gives a description of the dependency between entities,
    - Section 3.5, Data Dependencies: defines dependencies between the data entities,
    - Section 3.6, Interface Description: describes the interface of each of the functions,
    - Section 3.7, Detailed Design: details the algorithm and design for each function presented in Section 3.6,
    - Section 3.8, Operating System Services: describes operating system services used by entities,
    - Section 3.9, Error / Warning Handling: defines the handling of errors and warnings.

2 REFERENCES

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, 23 September 1998, www.ieee.org
IEEE Std 1558-2004	IEEE Standard for Software Documentation for Rail Equipment and Systems, 23 September 2004, www.ieee.org

Vapor Stone Rail Systems Documents

Internal Standards and Plans:

201111207	Software Project Management Plan, Denver Door System, Rev. 0
201111208	Software Quality Assurance Plan, Denver Door System, Rev. 1
201111186	Software Configuration Management Plan, Denver Door System, Rev. 1
201111254	Software Verification and Validation Plan, Denver Door System, Rev. 0
CMM-01	Software Engineering Process, Rev. 3, 2001-03-31
CMMI-02	Software Engineering Policy Manual, Rev. 1, 2008-07-18
S17-07	Software Design Document Template, Rev.1, 2003-02-19[1]

Input Documentation:

201111270	Generic DCU Application – Software Requirements Specifications (SRS), Rev. 1
201111489	Generic Software Documentation Project – Portable Test Interface Protocol, Interface Control Document (ICD), Rev. 1
201111576	Software Applications Configuration Specifications (SACS), Denver Door System, Rev. 1

Referenced Child Documentation:

201111544	Generic Software Documentation Project – Watchdog Module SDD, Rev. 0
201111554	Generic Software Documentation Project – Proportional Integral Derivative (PID) Module SDD, Rev. 0
201111568	Generic Software Documentation Project – Door Operator Module SDD, Rev. 1
201111581	Generic Software Documentation Project – Real-Time Clock (RTC) Module SDD, Rev. 1
201111582	Generic Software Documentation Project – Serial Driver Module SDD, Rev. 1
201111583	Generic Software Documentation Project – Fault Logger Module SDD, Rev. 1
201111584	Generic Software Documentation Project – Self-Test Module SDD, Rev. 1
201111585	Generic Software Documentation Project – Real-Time Data Monitoring (RTDM) Module SDD, Rev. 1
201111586	Generic Software Documentation Project – PTA Protocol Module SDD, Rev. 0
201111628	Denver – Input / Output Module SDD, Rev. 1
201111630	Denver – Monitoring Module SDD, Rev. 1

[1] Complemented with IEEE-1558:2004 requirements.

3 SOFTWARE DESIGN

3.1 Context Overview

The Figure 3-1 below provides an overview of the hardware context where the Door Control Unit Application is running.

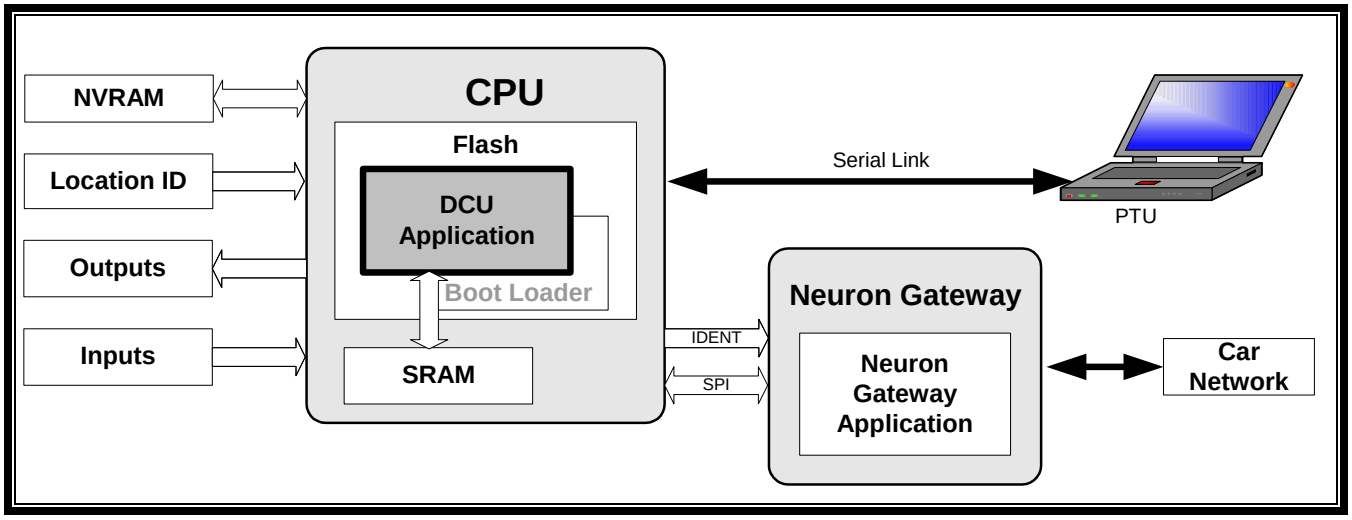

Figure 3-1: Hardware Block Diagram

3.2 Decomposition Description

The Figure 3-2 below provides a detailed view of the module entities involved in the DCU software. Sub-sections following Figure 3-2 provide an overview of the functions implemented in each of these modules.

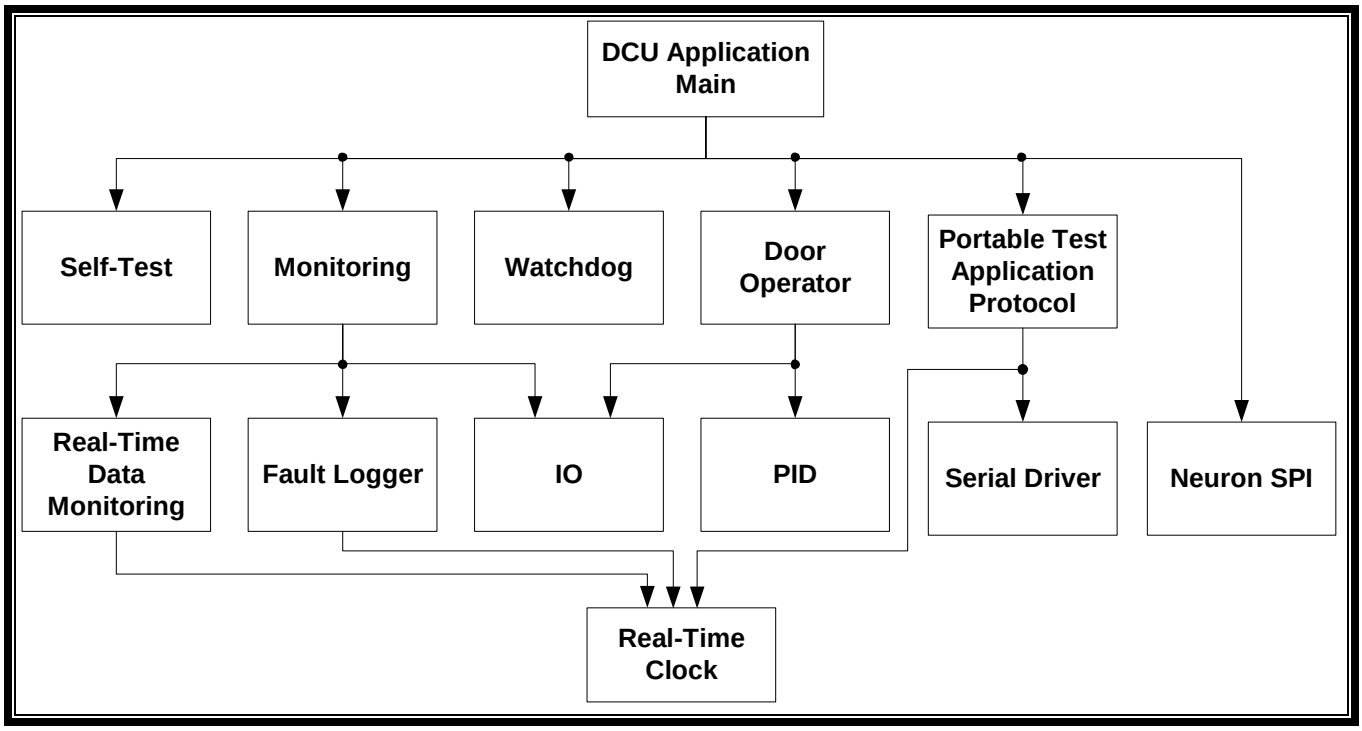

Figure 3-2: Hierarchical Diagram

3.2.1 Door Control Unit Application Main

The Main Module initializes the CPU and all its peripherals. It tests the memory and validates Boot Loader and main Application integrity. The Main Module reports any errors related to the peripherals. If no error is found, the main application starts the operating system and all tasks.

3.2.2 Self-Test Module

The Self-Test Module provides continuous self-testing functionally. It implements a task with a high priority but scheduled at a low periodicity that verifies the integrity of the memory and the program. Other modules, upon request can retrieve test results.

3.2.3 Monitoring Module

The Monitoring Module receives validated signals periodically from the IO Driver. Upon detection of errors, the Monitoring module logs the fault by calling the Fault Logger module and associates RTDM to the fault, if applicable. The monitoring module is also responsible for detecting door commands and sending them to the Door Operator Module (see 3.2.5).

3.2.4 Watchdog Module

The Watchdog Module monitors registered tasks and takes care of triggering the hardware watchdog circuit for preventing hardware reset of the running application. Whenever a registered task is not reporting within its configured period, the Watchdog Module stops triggering the hardware watchdog circuit to force a hardware reset.

3.2.5 Door Operator Module

The Door Operator module receives commands from the Monitoring module to open or close the door. It sends commands to the PID Module to execute the commands received. It also sets or resets outputs depending on the

received command. The Door Operator module receives sensors status (Solenoid Lock Sensing Switch and Panel Sensing Switch) from the IO Control module. It controls the position of the door and takes action from those sensors status.

3.2.6 Portable Test Application Protocol Module

The Portable Test Application (PTA) module receives commands from the Serial Port Driver regarding self-test, event log, real-time data monitoring. The PTA module sends the results of its command to the Serial Port Driver.

3.2.7 Real-Time Data Monitoring (RTDM) Module

The RTDM module captures all the relevant signals and puts them in a buffer. After an event has occurred, a command comes from the Monitoring module to save all the buffered data into the permanent memory and continue to save data as specified in the SACS document. This module also manages the snapshot files in the permanent memory.

3.2.8 Fault Logger Module

The Fault Logger module provides event-logging services. It provides event activations and event de-activations reporting for all other modules.

3.2.9 IO Control Module

The IO Control Module provides all the services to read from inputs and write to outputs. It also verifies and validates periodically the inputs read.

3.2.10 PID Module

The Proportional Integral Derivative (PID) Module implements all the functions to support the initialization, control, interrupt setting, and status report similar to those of the LM628/LM629 Motion Controller chip.

3.2.11 Serial Port Driver

The Serial Port Driver takes care of the input and output data on the serial port.

3.2.12 Real-Time Clock

The Real-Time Clock Module holds the time value used in faults and snapshots logging.

3.2.13 SPI Module

The SPI Module takes care of the input and output data on the Neuron SPI port.

3.3 Data Decomposition

The only data used exclusively by the main module is the DCU Application version information.

Each specific module SDD defines its shared data and module specific data.

3.4 Dependency Description

3.4.1 Inter-Entity Dependencies

The main thread starts all modules. Therefore, all modules depend on the main thread.

Specific module SDD defines the dependencies for each module. Please refer to the specific module SDD documents listed in Table 1-1 for more information about dependencies.

3.4.2 Inter-Process Dependencies

Multiple modules start their own thread/process for handling their functions. Figure 3-3 below provides a detailed view of all running threads and defines their communication channels.

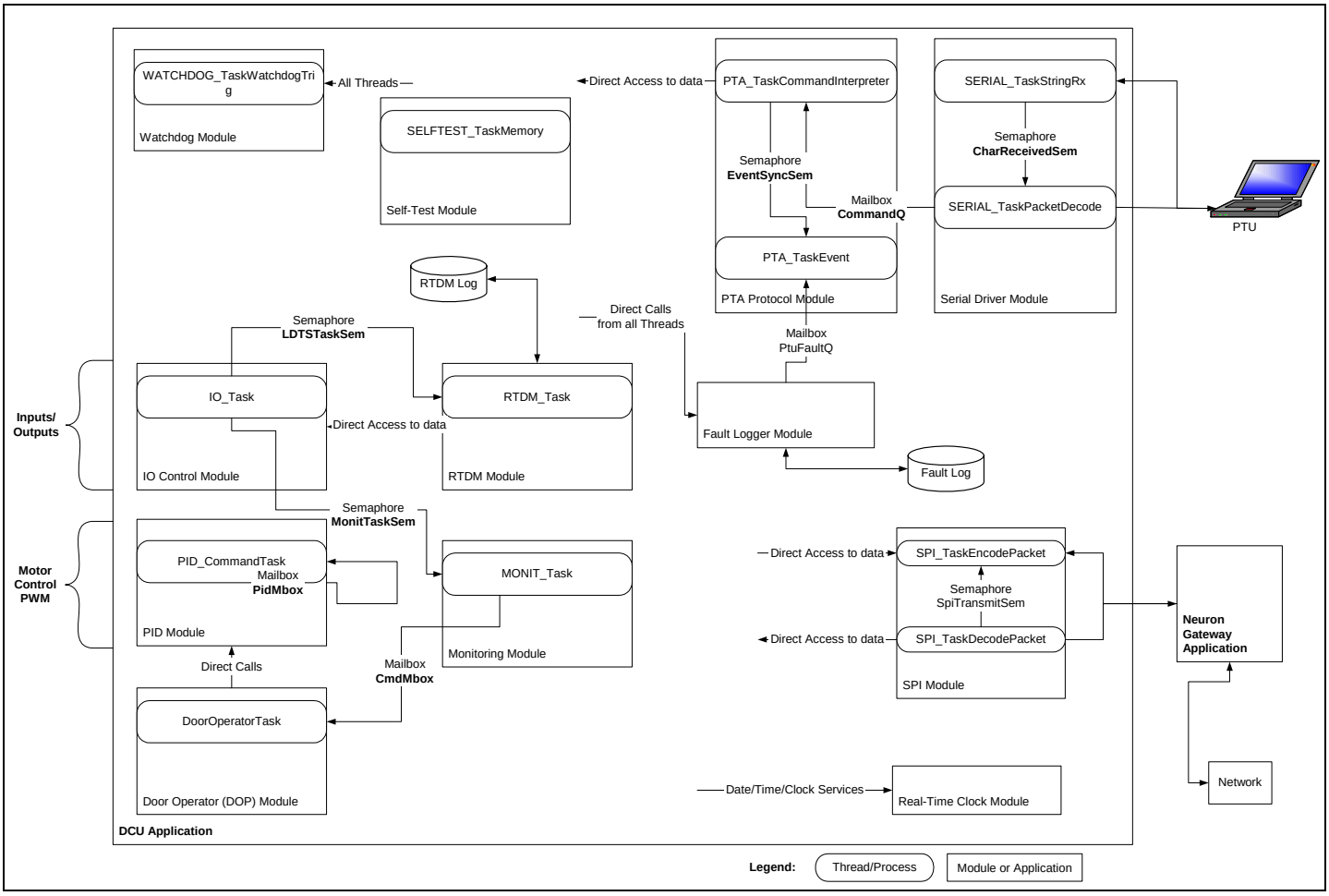

Figure 3-3: Inter-Process Dependencies Diagram

3.5 Data Dependencies

See Figure 3-3 for data dependencies.

3.6 Interface Description

3.6.1 Internal Interfaces

Please refer to the specific module SDD documents listed in Table 1-1 for more information about Interface definitions.

3.6.2 External Interfaces

The only functions documented in this SDD are the main() function and the BootTask() function presented below alphabetically ordered.

Function Name	void BootTask(void);
Description	This function is used to initialize all modules and start all threads defined in Figure 3-3. When all task starting is completed, this task kills itself and let other created tasks take the control of the application.
Input	None
Output	None
Returns	None
Called From	main()

Function Name	void main(void);
Description	This function is used to verify the memory and application integrity before starting the BootTask() function. In case of integrity failure, the DCU Application is not started and the system is reset.
Input	None
Output	None
Returns	None
Called From	CPU Initialization

3.6.3 Data Dictionary

Please refer to the specific module SDD documents listed in Table 1-1 for more information about Interface definitions.

3.7 Detailed Design

Algorithms related to starting module threads are defined in this document. All other algorithms are defined in their own module SDD. Please refer to the specific module SDD documents listed in Table 1-1 for more information about detailed function algorithms.

3.7.1 BootTask() Function

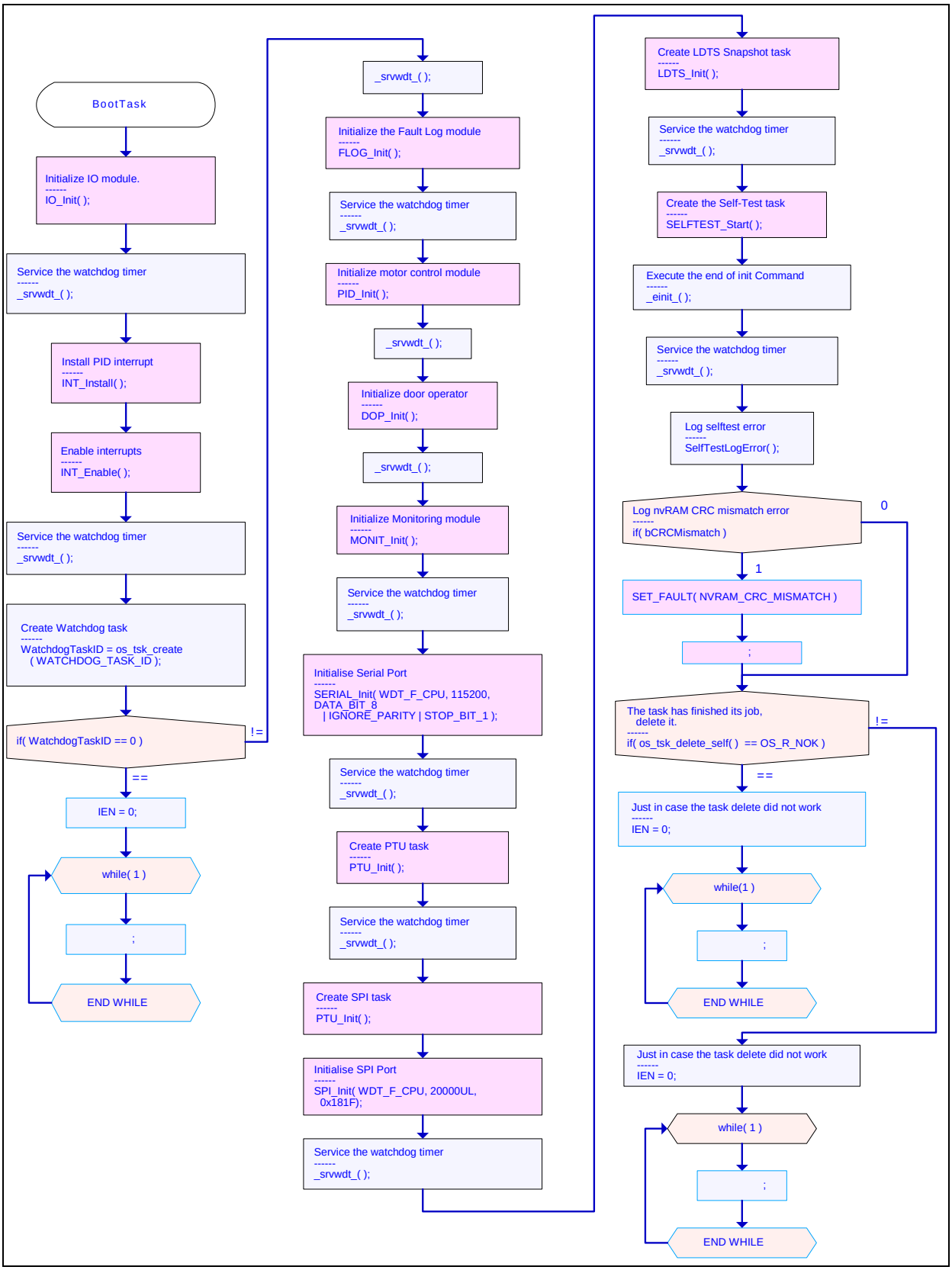

Figure 3-4: BootTask() Function Algorithm

3.7.2 main() Function

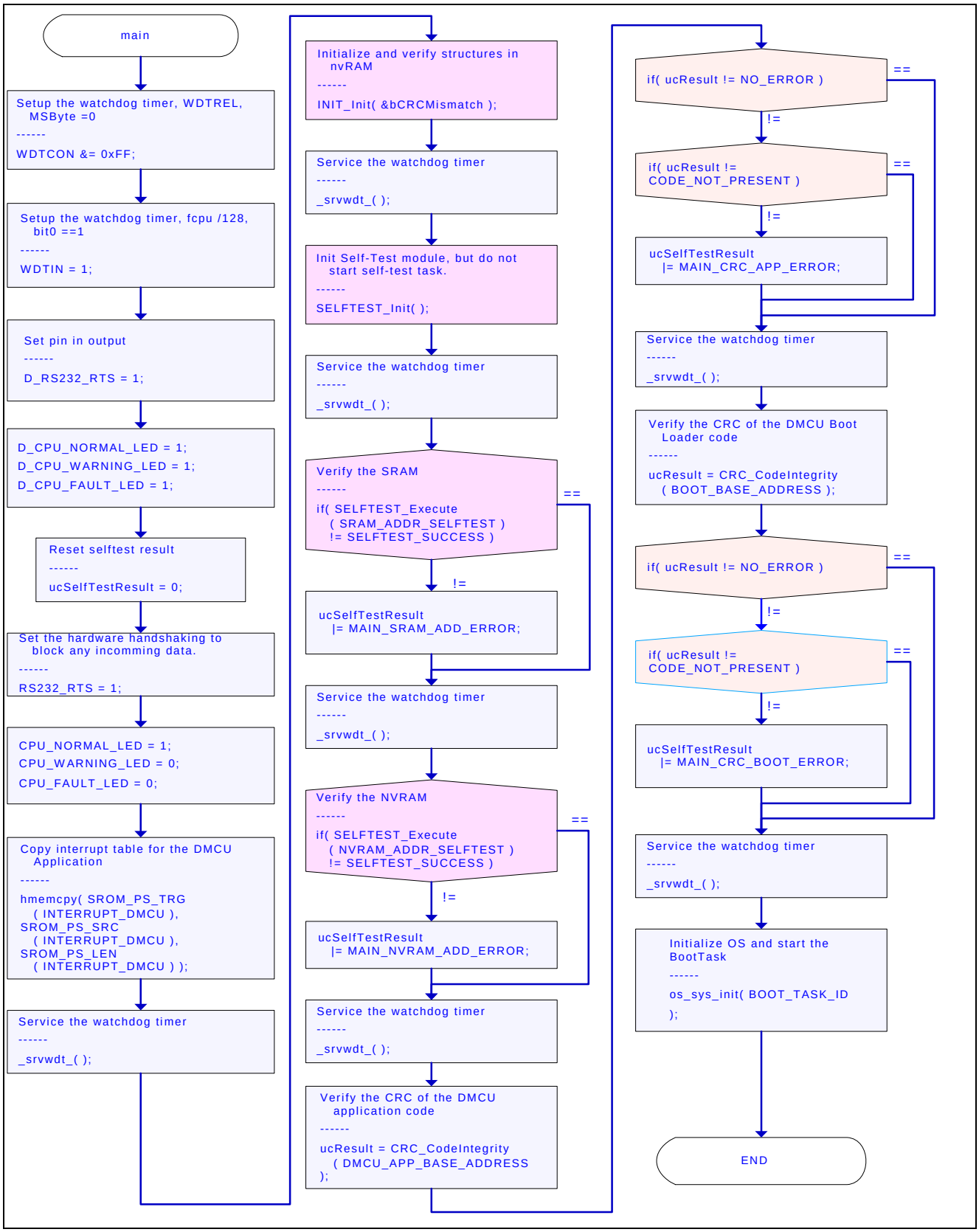

Figure 3-5: main() Function Algorithm

3.8 Operating System Services

The DCU Application uses Keil ARTX166 real-time operating system. The operating system services used by the DCU Application are:

- Task Manager
- Semaphores
- Mailboxes
- Delays/Timers

3.9 Error / Warning Handling

Please refer to the SACS document 201111576 for detailed specifications about error handling and events reported to the network and PTU interfaces.

The design of the monitoring module is project specific and is defined in the Monitoring Module SDD document 201111630.